Transactions and Concurrency Control

Keeping data correct when many users act at once

1. What is a Transaction?

A transaction is a logical unit of work made of one or more read and write operations that must be treated as a single, indivisible step. The classic example is a bank transfer: debit account A, then credit account B. Either both happen or neither does.

2. States of a Transaction

- **Active:** the transaction is executing its operations.
- **Partially committed:** the final operation has run, but the changes are not yet permanent.
- **Committed:** the changes are saved permanently.
- **Failed:** normal execution cannot continue.
- **Aborted:** the transaction is rolled back and the database is restored to its earlier state.

3. Schedules

A schedule is the order in which the operations of several transactions are executed. A **serial schedule** runs one transaction completely before the next begins; it is always correct but slow. A **concurrent schedule** interleaves operations for better throughput, which raises the risk of errors.

4. Problems of Uncontrolled Concurrency

Problem	What happens
Dirty read	T2 reads a value written by T1 that has not committed, and T1 later rolls back
Lost update	Two transactions read the same value and both write back, so one update overwrites the other
Unrepeatable read	T1 reads a row twice and gets different values because T2 changed it in between
Phantom read	T1 runs the same query twice and sees new rows that T2 inserted in between

5. The ACID Properties

- **Atomicity:** all of the transaction happens, or none of it.
- **Consistency:** a transaction takes the database from one valid state to another.
- **Isolation:** concurrent transactions do not see each other's partial results.
- **Durability:** committed changes survive crashes and power failures.

6. Serializability

A concurrent schedule is **serializable** if its result is equal to that of some serial schedule. It is the standard correctness test for concurrent execution.

- **Conflict serializability:** two operations conflict if they belong to different transactions, touch the same data item, and at least one is a write. If a schedule can be turned into a serial one by swapping only non-conflicting operations, it is conflict serializable.
- **View serializability:** a weaker condition that compares which transaction reads from which; every conflict serializable schedule is also view serializable, but not the other way round.

7. Precedence Graph Test

Draw one node per transaction, and add an edge Ti → Tj whenever an operation of Ti conflicts with a later operation of Tj. The schedule is conflict serializable **if and only if the graph has no cycle**. A topological order of the graph gives an equivalent serial order.

8. Lock-Based Protocols

- **Shared lock (S):** allows reading; many transactions can hold it on the same item together.
- **Exclusive lock (X):** allows reading and writing; no other lock is allowed on that item at the same time.
- **Two-Phase Locking (2PL):** each transaction first acquires locks (growing phase) and then only releases them (shrinking phase). 2PL guarantees conflict serializability.
- **Strict 2PL:** holds all exclusive locks until commit or abort, which also prevents cascading rollbacks.

9. Lock Compatibility

Held \ Requested	Shared (S)	Exclusive (X)
Shared (S)	Granted	Must wait
Exclusive (X)	Must wait	Must wait

10. SQL Isolation Levels

Level	Dirty read	Unrepeatable read	Phantom
Read Uncommitted	Possible	Possible	Possible
Read Committed	Prevented	Possible	Possible
Repeatable Read	Prevented	Prevented	Possible
Serializable	Prevented	Prevented	Prevented

11. Deadlock

A deadlock occurs when two or more transactions wait for each other's locks forever. For example, T1 holds a lock on A and waits for B, while T2 holds a lock on B and waits for A.

- **Detection:** build a wait-for graph with an edge Ti → Tj when Ti waits for Tj. A cycle means deadlock, and the DBMS aborts one victim transaction to break it.
- **Prevention, wait-die:** an older transaction may wait for a younger one, but a younger one that requests a lock held by an older one is aborted.
- **Prevention, wound-wait:** an older transaction aborts (wounds) a younger lock holder; a younger one waits for an older one.
- **Timeout:** a transaction that waits too long is simply aborted and restarted.

12. Timestamp Ordering

Each transaction receives a unique timestamp when it starts. Every data item records the largest read timestamp and write timestamp that touched it. An operation that would violate timestamp order is rejected and its transaction is rolled back and restarted with a new timestamp. This protocol uses no locks, so it can never deadlock.

13. Recovery from Failure

Recovery restores the database to a consistent state after a crash, using a **log** kept on stable storage.

- **Log record:** stores the transaction ID, the data item, its old value and its new value.
- **Write-Ahead Logging (WAL):** the log record must reach disk before the changed data page does.
- **Undo:** reverses the changes of transactions that had not committed at the time of the crash.
- **Redo:** reapplies the changes of committed transactions that may not have reached the disk.
- **Checkpoint:** a marker written periodically so recovery only has to scan the log from the last checkpoint.

14. Deferred vs Immediate Update

Technique	When the database is changed	On recovery
Deferred update	Only after the transaction commits	Redo committed transactions; no undo needed
Immediate update	While the transaction is still running	Undo uncommitted and redo committed ones

15. Shadow Paging

An alternative to logging: the DBMS keeps a *current page table* and a *shadow page table*. Writes go to new pages, and only at commit is the shadow table replaced by the current one. After a crash the old shadow table is still intact, so no undo or redo is needed.